

Systemy Sztucznej Inteligencji

Dokumentacja projektu: Wykorzystanie drzew decyzyjnych w klasyfikacji emocji

Adam Drażyk & Marta Załuska

14 czerwca 2026

Część I

Opis programu

Projekt polega na klasyfikacji emocji na podstawie zdjęć twarzy z wykorzystaniem autorskiej implementacji algorytmu Lasu Losowego (Random Forest). Przed analizą obrazów, program poddaje je ekstrakcji cech przy użyciu algorytmu HOG (Histogram of Oriented Gradients), co uodparnia model na zmiany oświetlenia i przesunięcia.

Instrukcja obsługi

Do uruchomienia programu wymagane jest środowisko Python oraz pakiety: `numpy`, `scikit-image`, `matplotlib`, `seaborn`, `scikit-learn` oraz `networkx`.

Struktura katalogów wymaga umieszczenia materiału badawczego w ścieżkach `Data/train` oraz `Data/test`, gdzie wewnątrz powinny znajdować się foldery odpowiadające poszczególnym klasom emocji (np. `angry`, `happy`).

Głównym punktem wejściowym aplikacji jest plik `main.py`. W jego nagłówku znajdują się trzy flagi sterujące działaniem programu:

- `PREPROCESSING` – włącza ekstrakcję cech HOG ze zdjęć i zapis do plików `.npy`.
- `TRAIN_MODEL` – uruchamia trenowanie lasu losowego. Jeśli jest ustawiona na `False`, program wczytuje gotowy model z folderu `Model/`.
- `VALIDATION_CURVE` – generuje wykres wpływu liczby estymatorów na skuteczność algorytmu.

Uruchomienie skryptu wykonuje się poleceniem: `python main.py`.

Dodatkowe informacje

Materiał badawczy stanowił zbiór FER-2013 pozyskany z bazy Kaggle. Składał się z 35900 zdjęć twarzy (proporcja zbioru treningowego do testowego to 80:20), każde zdjęcie miało wymiary 48×48 px i było udostępnione w skali szarości.



Rysunek 1: Przykładowe zdjęcie ze zbioru treningowego (klasa **sad**)

Do przetwarzania zastosowano optymalizację czasową poprzez zrównoleglenie obliczeń (wieloprocusowość za pomocą `concurrent.futures`), co pozwoliło na znacznie szybsze trenowanie pojedynczych drzew na wielu rdzeniach procesora.

Część II

Opis działania

Praca algorytmu na surowych pikselach jest wysoce nieefektywna, ponieważ wartość piksela mocno zależy od oświetlenia i cieni. Aby dokonać poprawnej klasyfikacji, dane przekształcono na reprezentację gradientową algorytmem HOG po uprzednim wyrównaniu histogramu (`exposure.equalize_adapthist`).

Niech $I(x, y)$ oznacza funkcję jasności piksela o współrzędnych x, y . Dla każdego piksela oblicza się gradienty pionowe i poziome reprezentujące różnicę jasności między sąsiadami:

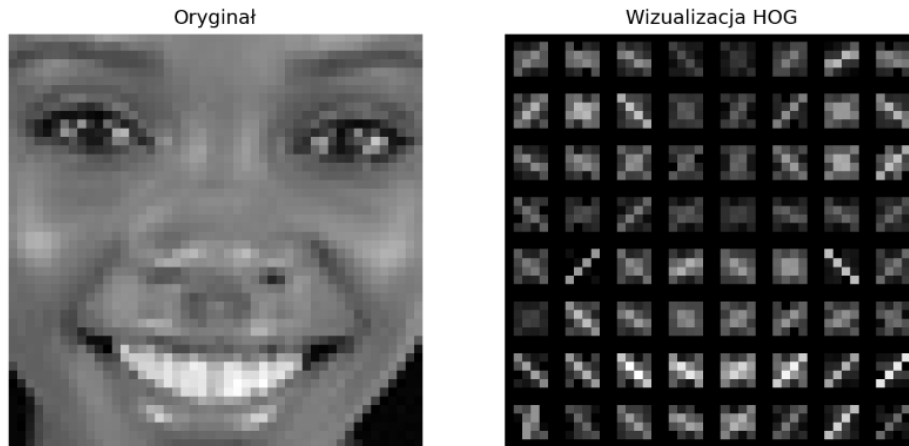
$$G_x(x, y) = I(x + 1, y) - I(x - 1, y) \quad (1)$$

$$G_y(x, y) = I(x, y + 1) - I(x, y - 1) \quad (2)$$

Następnie wyznaczane są dwa kluczowe parametry – magnituda (określająca siłę gradientu) oraz orientacja (odpowiadająca za kierunek krawędzi):

$$m(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2} \quad (3)$$

$$\theta(x, y) = \arctan \left(\frac{G_y(x, y)}{G_x(x, y)} \right) \quad (4)$$



Rysunek 2: Przykładowa wizualizacja HOG

Model Lasu Losowego wybrano ze względu na możliwość zrównoleglenia obliczeń i radzenie sobie z długimi wektorami cech. Przy każdym podziale węzła dla drzewa, sprawdzany jest podzbiór cech o rozmiarze $\sqrt{p}$. Wybór najlepszego podziału realizowany jest poprzez minimalizację entropii:

$$H(S) = - \sum_i p_i \log_2(p_i) \quad (5)$$

Algorytm

Poniżej przedstawiono zarys przepływu procesu przygotowania i predykcji.

Data: Zbiór zdjęć FER-2013, N estymatorów Lasu Losowego

Result: Wygenerowane metryki ewaluacyjne

Wczytaj mapowanie emocji (**angry**, **fear**, **happy**, **neutral**, **sad**, **surprise**);

if *PREPROCESSING* == *True* **then**

foreach *zdjęcie w zbiorze* **do**

 Wyrównaj histogram i znormalizuj obraz;

 Wyekstrahuj wektor HOG (8 orientacji, bloki 2×2);

end

 Zapisz macierze cech jako pliki *.npy*;

end

if *TRAIN_MODEL* == *True* **then**

 Zainicjuj *ProcessPoolExecutor* (wieloprocusowość);

foreach *drzewo z N* **do**

 Pobierz próbkę Bootstrap ze zbioru treningowego;

 Trenuj *DecisionTree* do osiągnięcia *max_depth*;

end

 Zapisz przetrenowany model (*pickle*);

end

Wykonaj predykcję na zbiorze *X_test* przez głosowanie większościowe wszystkich drzew;

Wygeneruj macierz pomyłek i tabele metryk z modułu *metrics.py*;

Algorithm 1: Główny proces działania aplikacji.

Zbiór danych

Oryginalny zbiór z Kaggle dzielił się na 7 klas, jednak siódma (*disgust*) była na tyle mała, że zaburzała wyniki – w związku z tym została pominięta. Klasy przyporządkowano do następujących wartości w module *main.py*:

- *angry* $\mapsto 0$
- *fear* $\mapsto 1$
- *happy* $\mapsto 2$
- *neutral* $\mapsto 3$
- *sad* $\mapsto 4$
- *surprise* $\mapsto 5$

Implementacja

Aplikacja została podzielona na następujące pliki:

- *main.py* – plik startowy organizujący przepływ danych.

- `preprocessing.py` – wczytywanie obrazów i realizacja algorytmu ekstrakcji HOG.
 - Funkcja `preprocess_data(emotions_map, path)` – przygotowanie zdjęć w folderach treningowym i testowym do dalszej analizy
 - Funkcja `preprocess_image(image_path)` – normalizacja zdjęcia do skali szarości i nadanie każdemu pikselowi wartości w przedziale $[0, 1]$ na podstawie jasności
 - Funkcja `extract_features(image)` – wywołanie ekstrakcji HOG dla danego zdjęcia o przyjętych parametrach
- `random_forest.py` – autorska implementacja algorytmu Random Forest wraz z węzłami drzew decyzyjnych z użyciem wieloprocusowości.
 - Klasa `TreeNode` – pojedynczy węzeł drzewa z przypisaną cechą i wartością podziału
 - Klasa `DecisionTree` – drzewo decyzyjne, które wybiera najlepszy podział na podstawie entropii cech
 - Klasa `RandomForest` – las losowy, tworzący i trenujący odpowiednią ilość drzew, implementujący Bootstrapping oraz przeprowadzający głosowanie większościowe
- `validation_curve.py` – generowanie krzywej walidacji dla różnej ilości drzew.
- `metrics.py` – własne funkcje do liczenia metryk (Accuracy, Recall itp.) oraz wizualizacja macierzy za pomocą biblioteki `seaborn`.
 - Funkcja `plot_confusion_matrix(y_true, y_pred, class_names)` – generowanie zbiorczej macierzy pomyłek dla wszystkich klas 4
 - Funkcja `plot_single_class_cm(TP, TN, FP, FN, class_name)` – generowanie macierzy pomyłek dla jednej, podanej klasy
 - Funkcja `run_metrics(y_true, y_pred, class_names)` – obliczanie metryk, czyli wartości Micro i Macro wskaźników (Accuracy, Recall, Precision, F1 Score, Specificity)
 - Funkcja `plot_metrics_table(table_data, class_names)` – generowanie tabeli z metrykami dla każdej z klas 3
- `graphics.py` – generowanie obrazów przykładowych po transformacji HOG 2

Eksperymenty i Testy

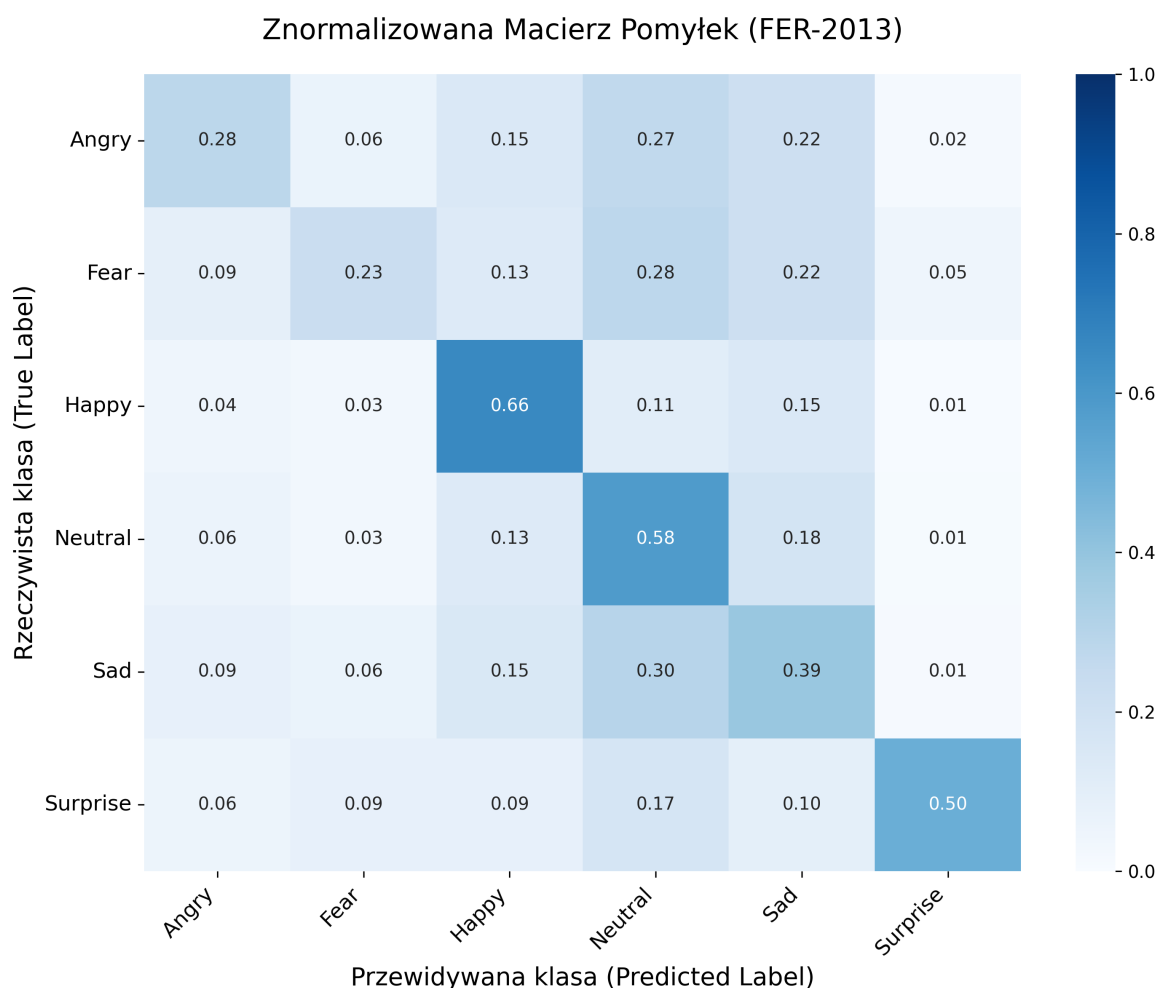
Zbiór FER-2013 uchodzi za trudny – skuteczność ludzka przy oznaczaniu wynosi zaledwie 65.5%, a zbiór posiada wiele błędnych etykiet. Uzyskano globalne parametry:

- Accuracy: 81.54%
- Recall/Precision/F1 Score: 44.63%
- Specificity: 88.93%

Wbudowane metody pozwoliły wygenerować wizualną tabelę metryk w podejściu One-vs-Rest dla wszystkich sześciu klas (z najwyższymi wynikami F1 dla klasy **happy** oraz **surprise**).

	Accuracy	Recall	Precision	F1 Score	Specificity
ANGRY	83.47%	28.50%	42.66%	34.17%	93.21%
FEAR	83.09%	23.44%	45.11%	30.85%	94.53%
HAPPY	83.29%	65.70%	50.25%	56.95%	86.85%
NEUTRAL	73.38%	58.39%	37.87%	45.95%	76.98%
SAD	74.10%	39.05%	35.42%	37.15%	82.64%
SURPRISE	91.94%	50.18%	80.81%	61.92%	98.21%

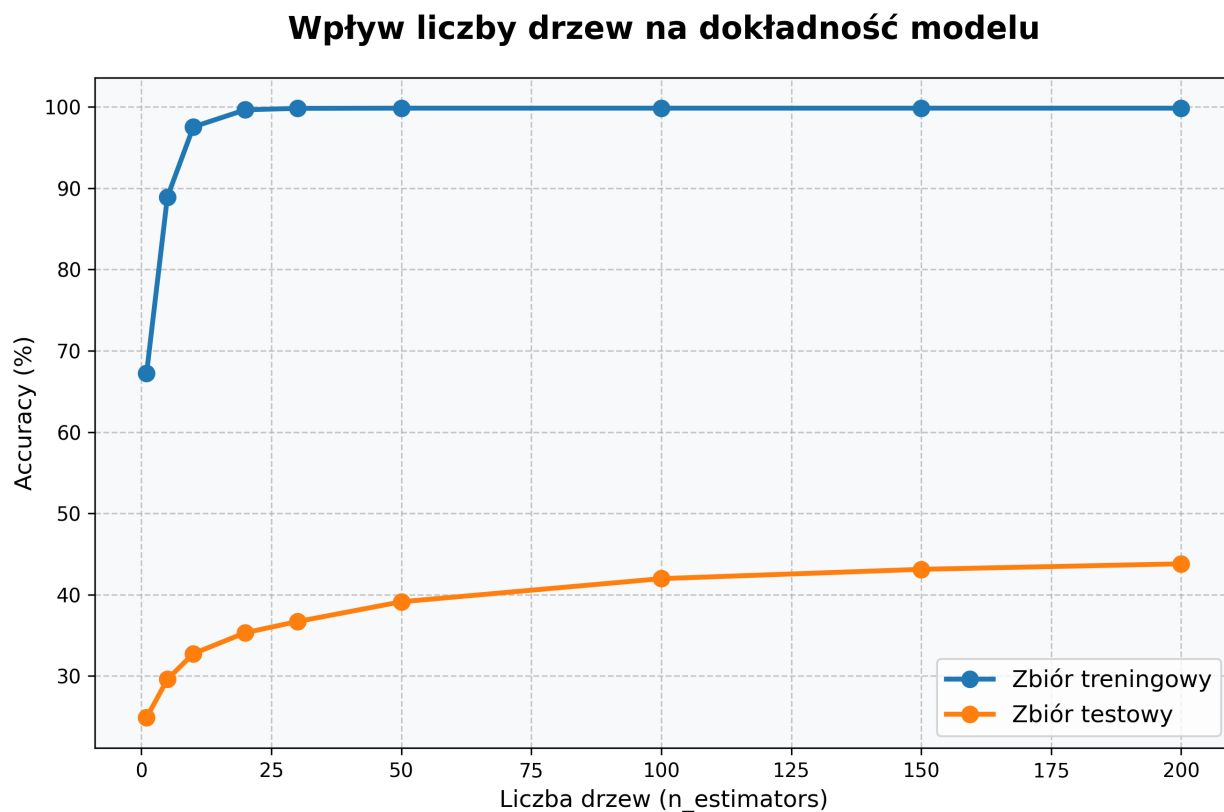
Rysunek 3: Tabela metryk dla każdej z klas



Rysunek 4: Macierz błędów dla poszczególnych klas

W module `validation_curve.py` przetestowano wpływ liczby drzew (zmienna `n_estimators` od 1 do 200) na końcową skuteczność. Zaobserwowano, że dokładność testowa stabilizuje się

przy wartości w okolicach 50 estymatorów (nieco ponad 40%), a dalsze dodawanie drzew nie chroni algorytmu płytkiego uczenia przed nasyceniem.



Rysunek 5: Krzywa walidacji dla stworzonego modelu

Pełen kod aplikacji

main.py

```
1 import os
2 import pickle
3 import numpy as np
4
5 from preprocessing import preprocess_data
6 from metrics import plot_confusion_matrix, run_metrics
7 from random_forest import RandomForest
8 from validation_curve import plot_tree_validation_curve
9
10 # ----- MODEL SETTINGS -----
11 PREPROCESSING = False
12 TRAIN_MODEL = False
13 VALIDATION_CURVE = True
14
15 # ----- CMD COLOR SETTINGS -----
16 RED = '\033[91m \x1B[3m'
17 YELLOW = '\033[93m \x1B[3m'
18 GREEN = '\033[92m \x1B[3m'
19 RESET = '\033[0m \x1B[0m'
20
21 # ----- PATHS TO DATA -----
22 TRAIN_PATH = os.path.join("Data", "train")
23 TEST_PATH = os.path.join("Data", "test")
24 MODEL_FILE = os.path.join("Model", "saved_rf_model.pkl")
25
26 #
27 -----
28
29 if __name__ == '__main__':
30     emotions_map = {"angry": 0, "fear": 1, "happy": 2,
31                     "neutral": 3, "sad": 4, "surprise": 5}
32
33     if PREPROCESSING:
34         print(f"{GREEN}Rozpoczynam ekstrakcj cech HOG...{RESET}")
35         train_features, train_labels = preprocess_data(emotions_map,
36                                                         TRAIN_PATH)
37         test_features, test_labels = preprocess_data(emotions_map,
38                                                         TEST_PATH)
39         print(f"{GREEN}Ekstrakcja cech zakończona pomylnie.{RESET}")
40
41         print(f"{GREEN}Generuj macierze cech HOG i wektory etykiet...{RESET}")
42
43         X_train = np.array(train_features, dtype=np.float32)
44         y_train = np.array(train_labels, dtype=np.float32)
45         X_test = np.array(test_features, dtype=np.float32)
46         y_test = np.array(test_labels, dtype=np.float32)
47         print(f"{GREEN}Pomylnie stworzono i zapisano pliki.{RESET}")
48
49         print(f"{GREEN}Zapisuj dane do pliku...{RESET}")
50         np.save(os.path.join("Model", "X_train.npy"), X_train)
```

```

46     np.save(os.path.join("Model", "y_train.npy"), y_train)
47     np.save(os.path.join("Model", "X_test.npy"), X_test)
48     np.save(os.path.join("Model", "y_test.npy"), y_test)
49     print(f"{GREEN}Dane zapisane pomy lnie.{RESET}")
50 else:
51     print(f"{YELLOW}Wczytuj dane z pliku...{RESET}")
52     try:
53         X_train = np.load(os.path.join("Model", "X_train.npy"))
54         y_train = np.load(os.path.join("Model", "y_train.npy"))
55         X_test = np.load(os.path.join("Model", "X_test.npy"))
56         y_test = np.load(os.path.join("Model", "y_test.npy"))
57
58     except FileNotFoundError:
59         print(f"{RED}Nie znaleziono plik w , wygeneruj dane na nowo
60             .{RESET}")
61         quit()
62     print(f"{YELLOW}Dane wczytano pomy lnie.{RESET}")
63
64 if TRAIN_MODEL:
65     print(f"{GREEN}Rozpoczynam klasyfikacj lasem losowym...{RESET}
66         ")
67     random_forest = RandomForest(500, 15)
68     random_forest.train(X_train, y_train)
69     print(f"{GREEN}Klasyfikacja zako czona pomy lnie.{RESET}")
70     print(f"{GREEN}Zapisuj wytrenowany model do pliku...{RESET}")
71     with open(MODEL_FILE, 'wb') as file:
72         pickle.dump(random_forest, file)
73     print(f"{GREEN}Model zapisany.{RESET}")
74 else:
75     print(f"{YELLOW}Wczytuj gotowy model z dysku...{RESET}")
76     with open(MODEL_FILE, 'rb') as file:
77         random_forest = pickle.load(file)
78     print(f"{YELLOW}Model wczytany pomy lnie.{RESET}")
79
80 predictions = random_forest.predict(X_test)
81
82 print(f"{GREEN}Generuj macierz pomy ek...{RESET}")
83 emotions_list = ["Angry", "Fear", "Happy", "Neutral", "Sad", "
84     Surprise"]
85 plot_confusion_matrix(y_test, predictions, emotions_list)
86 print(f"{GREEN}Macierz wygenerowana pomy lnie.{RESET}")
87
88 print(f"{GREEN}Przygotowuj metryki modelu...{RESET}")
89 run_metrics(y_test, predictions, emotions_list)
90 print(f"{GREEN}Wypisywanie metryk zako czone.{RESET}")
91
92 if VALIDATION_CURVE:
93     print(f"{GREEN}Generowanie krzywej walidacji...{RESET}")
94     plot_tree_validation_curve(X_train, y_train, X_test, y_test)
95     print(f"{GREEN}Krzywa walidacji wygenerowana pomy lnie.{RESET}
96         ")

```

preprocessing.py

```
1 import os
2 from skimage import exposure
3 from skimage import io
4 from skimage.feature import hog
5 import numpy as np
6
7 RED = '\033[91m \x1B[3m'
8 YELLOW = '\033[93m \x1B[3m'
9 GREEN = '\033[92m \x1B[3m'
10 RESET = '\033[0m \x1B[0m'
11
12 def extract_features(image):
13     return hog(image, orientations=8, pixels_per_cell=(6, 6),
14                cells_per_block=(2, 2), transform_sqrt=True,
15                feature_vector=True)
16
17 def preprocess_image(image_path):
18     image = io.imread(image_path, as_gray=True)
19     image = exposure.equalize_adapthist(image, clip_limit=0.03)
20     image = image / 255.0
21     return image
22
23 def preprocess_data(emotions_map, path):
24     features = []
25     labels = []
26     for emotion_name, emotion_label in emotions_map.items():
27         emotion_path = os.path.join(path, emotion_name)
28
29         if not os.path.exists(emotion_path):
30             print(f"{YELLOW}Nie znalezione folderu {emotion_path}.{RESET}")
31             continue
32
33         for filename in os.listdir(emotion_path):
34             if filename.endswith(".jpg"):
35                 image_path = os.path.join(emotion_path, filename)
36
37                 processed_image = preprocess_image(image_path)
38                 features.append(extract_features(processed_image))
39                 labels.append(emotion_label)
40
41         print(f"{GREEN}Przetworzono klas {emotion_name} ({path[5:]}){RESET}")
42
43     return features, labels
```

random_forest.py

```
1 import random
2 import numpy as np
```

```

3 from networkx.generators import trees
4 import concurrent.futures
5
6 # ----- CMD COLOR SETTINGS -----
7 RED = '\033[91m \x1B[3m'
8 YELLOW = '\033[93m \x1B[3m'
9 GREEN = '\033[92m \x1B[3m'
10 RESET = '\033[0m \x1B[0m'
11
12 class TreeNode:
13     def __init__(self, feature_idx=None, feature_val=None,
14                  predicted_class=None):
15         self.feature_idx = feature_idx
16         self.feature_val = feature_val
17         self.left = None
18         self.right = None
19         self.predicted_class = predicted_class
20
21 class DecisionTree:
22     def __init__(self, max_depth=3, min_samples_leaf=5):
23         self.max_depth = max_depth
24         self.min_samples_leaf = min_samples_leaf
25         self.tree = None
26
27     def calculate_entropy_for_data(self, labels):
28         total_samples = len(labels)
29         if total_samples == 0:
30             return 0
31         _, counts = np.unique(labels, return_counts=True)
32         probabilities = counts / total_samples
33         return -np.sum(probabilities * np.log2(probabilities))
34
35     def split(self, data, feature_idx, feature_val):
36         mask = data[:, feature_idx] < feature_val
37         left_split = data[mask]
38         right_split = data[~mask]
39         return left_split, right_split
40
41     def find_best_split(self, data):
42         best_entropy = float('inf')
43         best_feature_idx = None
44         best_feature_val = None
45         best_left = None
46         best_right = None
47
48         n_features = data.shape[1] - 1
49         max_features = int(np.sqrt(n_features))
50         feature_indices = np.random.choice(n_features, size=max_features,
51                                           replace=False)
52
53         for idx in feature_indices:
54             thresholds = np.percentile(data[:, idx], [20, 40, 60, 80])
55             thresholds = np.unique(thresholds)
56
57             for val in thresholds:

```

```

56         left_split, right_split = self.split(data, idx, val)
57
58         if len(left_split) == 0 or len(right_split) == 0:
59             continue
60
61         n_total = len(data)
62         weight_left = len(left_split) / n_total
63         weight_right = len(right_split) / n_total
64
65         entropy_left = self.calculate_entropy_for_data(
66             left_split[:, -1])
67         entropy_right = self.calculate_entropy_for_data(
68             right_split[:, -1])
69
70         current_entropy = (weight_left * entropy_left) + (
71             weight_right * entropy_right)
72
73         if current_entropy < best_entropy:
74             best_entropy = current_entropy
75             best_feature_idx = idx
76             best_feature_val = val
77             best_left = left_split
78             best_right = right_split
79
80         return best_left, best_right, best_feature_idx, best_feature_val
81
82     def get_majority_class(self, labels):
83         values, counts = np.unique(labels, return_counts=True)
84         majority_idx = np.argmax(counts)
85         return values[majority_idx]
86
87     def create_tree(self, data, current_depth):
88         labels = data[:, -1]
89
90         if len(np.unique(labels)) == 1 or current_depth >= self.
91             max_depth or len(labels) <= self.min_samples_leaf:
92             return TreeNode(predicted_class=self.get_majority_class(
93                 labels))
94
95         left_data, right_data, split_idx, split_val = self.
96             find_best_split(data)
97
98         if left_data is None or right_data is None:
99             return TreeNode(predicted_class=self.get_majority_class(
100                 labels))
101
102         node = TreeNode(feature_idx=split_idx, feature_val=split_val)
103         node.left = self.create_tree(left_data, current_depth + 1)
104         node.right = self.create_tree(right_data, current_depth + 1)
105
106         return node
107
108     def train(self, X, y):
109         train_data = np.concatenate((X, y.reshape(-1, 1)), axis=1, dtype
110             =np.float32)

```

```

1103         self.tree = self.create_tree(data=train_data, current_depth=0)
1104
1105     def predict_one_sample(self, X_sample):
1106         node = self.tree
1107         while node.predicted_class is None:
1108             if X_sample[node.feature_idx] < node.feature_val:
1109                 node = node.left
1110             else:
1111                 node = node.right
1112         return node.predicted_class
1113
1114     def predict(self, X_set):
1115         predictions = [self.predict_one_sample(sample) for sample in
1116                        X_set]
1117         return np.array(predictions)
1118
1119 class RandomForest:
1120     def __init__(self, n_estimators=10, max_depth=3, min_samples_leaf=2):
1121         :
1122         self.n_estimators = n_estimators
1123         self.max_depth = max_depth
1124         self.min_samples_leaf = min_samples_leaf
1125         self.trees = []
1126
1127     def bootstrap_sample(self, X_train, y_train):
1128         N = X_train.shape[0]
1129         indices = np.random.choice(N, N)
1130         return X_train[indices], y_train[indices]
1131
1132     def train_single_tree(self, X, y):
1133         X_sample, y_sample = self.bootstrap_sample(X, y)
1134
1135         tree = DecisionTree(
1136             max_depth=self.max_depth,
1137             min_samples_leaf=self.min_samples_leaf
1138         )
1139         tree.train(X_sample, y_sample)
1140         return tree
1141
1142     def train(self, X_train, y_train, n_jobs=5):
1143         i = 1
1144         print(f"{GREEN}Rozpoczynam trenowanie...{RESET}")
1145         with concurrent.futures.ProcessPoolExecutor(max_workers=n_jobs)
1146             as executor:
1147             futures = [executor.submit(self.train_single_tree, X_train,
1148                                     y_train)
1149                        for _ in range(self.n_estimators)]
1150
1151             for future in concurrent.futures.as_completed(futures):
1152                 trained_tree = future.result()
1153                 self.trees.append(trained_tree)
1154                 print(f"{GREEN}Wytrenowano drzewo {i}/{self.n_estimators}
1155                       }.{RESET}")
1156                 i += 1

```

```

153     def predict(self, X):
154         print(f"{GREEN}Rozpoczynam predykcj ...{RESET}")
155         tree_preds = np.array([tree.predict(X) for tree in self.trees])
156         tree_preds = np.swapaxes(tree_preds, 0, 1)
157
158         final_predictions = []
159         for sample_votes in tree_preds:
160             values, counts = np.unique(sample_votes, return_counts=True)
161             majority_class = values[np.argmax(counts)]
162             final_predictions.append(majority_class)
163         print(f"{GREEN}Predykcja zakończona.{RESET}")
164         return np.array(final_predictions)

```

metrics.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4  from sklearn.metrics import confusion_matrix, classification_report
5
6  RED = '\033[91m \x1B[3m'
7  YELLOW = '\033[93m \x1B[3m'
8  GREEN = '\033[92m \x1B[3m'
9  RESET = '\033[0m \x1B[0m'
10
11 def plot_confusion_matrix(y_true, y_pred, class_names):
12     cm = confusion_matrix(y_true, y_pred)
13     cm_normalized = cm.astype('float') / (cm.sum(axis=1)[:, np.newaxis]
14     + 1e-9)
15
16     plt.figure(figsize=(10, 8))
17     sns.heatmap(cm_normalized,
18                 annot=True,
19                 fmt=".2f",
20                 cmap="Blues",
21                 xticklabels=class_names,
22                 yticklabels=class_names,
23                 vmin=0,
24                 vmax=1)
25
26     plt.title('Znormalizowana Macierz Pomyłek (FER-2013)', fontsize=16,
27             pad=20)
28     plt.ylabel('Rzeczywista klasa (True Label)', fontsize=14)
29     plt.xlabel('Przewidywana klasa (Predicted Label)', fontsize=14)
30
31     plt.xticks(rotation=45, ha='right', fontsize=12)
32     plt.yticks(rotation=0, fontsize=12)
33
34     plt.tight_layout()
35     plt.savefig('confusion_matrix_rf.png', dpi=300)
36     plt.show()
37
38 def accuracy(TP, TN, FP, FN):

```

```

37     return (TP + TN) / (TP + FP + FN + TN) if (TP + FP + FN + TN) > 0
        else 0
38
39 def sensitivity_recall(TP, TN, FP, FN):
40     return (TP) / (TP + FN) if (TP + FN) > 0 else 0
41
42 def presision(TP, TN, FP, FN):
43     return (TP) / (TP + FP) if (TP + FP) > 0 else 0
44
45 def f1_score(TP, TN, FP, FN):
46     return (2*TP) / (2*TP + FP + FN) if (2*TP + FP + FN) > 0 else 0
47
48 def specificity(TP, TN, FP, FN):
49     return (TN) / (TN + FP) if (TN + FP) > 0 else 0
50
51 def plot_single_class_cm(TP, TN, FP, FN, class_name):
52     cm_2x2 = np.array([[TN, FP],
53                        [FN, TP]])
54
55     annot_data = np.array([[f"TN{cm_2x2[0, 0]}", f"FP{cm_2x2[0, 1]}",
56                           [f"FN{cm_2x2[1, 0]}", f"TP{cm_2x2[1, 1]}"]]])
57
58     plt.figure(figsize=(4, 3))
59
60     sns.heatmap(cm_2x2,
61                 annot=annot_data,
62                 fmt='',
63                 cmap="Blues",
64                 cbar=False,
65                 annot_kws={"size": 10},
66                 xticklabels=['INNA', class_name.upper()],
67                 yticklabels=['INNA', class_name.upper()])
68
69     plt.title(f'Klasa: {class_name.upper()}', fontsize=12)
70     plt.ylabel('Rzeczywista klasa (Actual)', fontsize=10)
71     plt.xlabel('Przewidywana klasa (Predicted)', fontsize=10)
72
73     plt.xticks(fontsize=9)
74     plt.yticks(rotation=0, fontsize=9)
75
76     plt.tight_layout()
77     plt.show()
78
79 def run_metrics(y_true, y_pred, class_names):
80     cm = confusion_matrix(y_true, y_pred)
81     total_samples = cm.sum()
82     metrics = {accuracy: 'Accuracy', sensitivity_recall: 'Recall',
83               presision: 'Precision', f1_score: 'F1 Score', specificity
84               : 'Specificity'}
85
86     total_TP, total_FP, total_TN, total_FN = 0, 0, 0, 0
87     table_data = []
88
89     for i, class_name in enumerate(class_names):
90         TP = cm[i, i]

```

```

90     FP = cm[:, i].sum() - TP
91     FN = cm[i, :].sum() - TP
92     TN = total_samples - (TP + FP + FN)
93
94     total_TP += TP
95     total_FP += FP
96     total_FN += FN
97     total_TN += TN
98
99     row_data = [class_name.upper()]
100    for metric, _ in metrics.items():
101        row_data.append(metric(TP, TN, FP, FN) * 100)
102    table_data.append(row_data)
103
104    plot_single_class_cm(TP, TN, FP, FN, class_name)
105
106    plot_metrics_table(table_data, class_names)
107
108    print("OG    EM (Warto ci Micro):")
109    print(f"    Sumaryczne TP: {total_TP}")
110    print(f"    Sumaryczne FP: {total_FP}")
111    print(f"    Sumaryczne TN: {total_TN}")
112    print(f"    Sumaryczne FN: {total_FN}")
113
114    print("Globalne wska niki (Micro-Averaged):")
115    for metric, metric_str in metrics.items():
116        score = metric(total_TP, total_TN, total_FP, total_FN)
117        print(f"    {metric_str:<12} = {score * 100:.2f}%")
118
119    def plot_metrics_table(table_data, class_names):
120        columns = ['Accuracy', 'Recall', 'Precision', 'F1 Score', '
121                    Specificity']
122
123        cell_text = []
124        for row in table_data:
125            cell_text.append([f"{val:.2f}%" for val in row[1:]])
126
127        fig, ax = plt.subplots(figsize=(10, 4))
128        ax.set_axis_off()
129
130        table = ax.table(
131            cellText=cell_text,
132            rowLabels=[name.upper() for name in class_names],
133            colLabels=columns,
134            cellLoc='center',
135            loc='center',
136            colColours=['#e1e6f0'] * len(columns),
137            rowColours=['#e1e6f0'] * len(class_names)
138        )
139
140        table.auto_set_font_size(False)
141        table.set_fontsize(12)
142        table.scale(1, 2)
143
144        for (row, col), cell in table.get_celld().items():

```

```

144         if row == 0 or col == -1:
145             cell.set_text_props(weight='bold')
146
147     plt.title('Szczeg  owe Metryki Klasyfikacji (One-vs-Rest)',
148             fontsize=16, weight='bold', pad=20)
149     plt.tight_layout()
150     plt.savefig('metrics_table_rf.png', dpi=300, bbox_inches='tight')
151     plt.show()

```

validation_curve.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import time
4  from random_forest import RandomForest
5
6  RED = '\033[91m \x1B[3m'
7  YELLOW = '\033[93m \x1B[3m'
8  GREEN = '\033[92m \x1B[3m'
9  RESET = '\033[0m \x1B[0m'
10
11 def plot_tree_validation_curve(X_train, y_train, X_test, y_test):
12     tree_counts = [1, 5, 10, 20, 30, 50, 100, 150, 200]
13
14     train_scores = []
15     test_scores = []
16
17     for n in tree_counts:
18         print(f"{GREEN}Trenowanie Lasu Losowego dla n_estimators = {n}
19             ...{RESET}")
20
21         model = RandomForest(n_estimators=n, max_depth=15)
22         model.train(X_train, y_train)
23
24         y_train_pred = model.predict(X_train)
25         y_test_pred = model.predict(X_test)
26
27         train_acc = np.mean(y_train_pred == y_train) * 100
28         test_acc = np.mean(y_test_pred == y_test) * 100
29
30         train_scores.append(train_acc)
31         test_scores.append(test_acc)
32
33     plt.figure(figsize=(9, 6))
34
35     plt.plot(tree_counts, train_scores, 'o-', color="#1f77b4", label="
36         Zbi r treningowy", linewidth=2.5, markersize=8)
37     plt.plot(tree_counts, test_scores, 'o-', color="#ff7f0e", label="
38         Zbi r testowy", linewidth=2.5, markersize=8)
39
40     plt.title("Wp yw liczby drzew na dok adno  modelu", fontsize
41             =16, weight='bold', pad=20)
42     plt.xlabel("Liczba drzew (n_estimators)", fontsize=12)

```

```

39     plt.ylabel("Accuracy (%)", fontsize=12)
40
41     plt.legend(loc="lower right", fontsize=12)
42     plt.grid(True, linestyle='--', alpha=0.7)
43     plt.gca().set_facecolor('#f8f9fa')
44
45     plt.tight_layout()
46     plt.savefig('validation_curve_trees.png', dpi=300)
47     plt.show()

```

graphics.py

```

1  import os
2  from skimage.feature import hog
3  from skimage import io
4  import matplotlib.pyplot as plt
5  import numpy as np
6
7  IMAGE_PATH = os.path.join("Data", "test", "happy", "PrivateTest_2626531.
    jpg")
8
9  image = io.imread(IMAGE_PATH)
10
11  fd, hog_image = hog(image, orientations=8, pixels_per_cell=(6, 6),
12                      cells_per_block=(2, 2), visualize=True)
13
14  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5), sharex=True,
    sharey=True)
15
16  ax1.axis('off')
17  ax1.imshow(image, cmap=plt.cm.gray)
18  ax1.set_title('Orygina ')
19
20  hog_image_rescaled = np.power(hog_image, 1/2)
21
22  ax2.axis('off')
23  ax2.imshow(hog_image_rescaled, cmap=plt.cm.gray)
24  ax2.set_title('Wizualizacja HOG')
25
26  plt.show()

```
